

Coding Project Final Report



Greedy Dwarf

Prepared by Brian Rosca, Victoria Rossi, Hristian Tountchev, Shaan Kohli

Table of Contents

I Project Description	4
1 Project Overview.....	4
2 Project Domain	4
3 Relationship to Other Documents	5
4 Naming Conventions and Definitions	6
II Project Deliverables	10
5 First Release	10
6 Second Release	12
7 Comparison with Original Project Design Document	15
III Testing.....	17
8 Items to be Tested.....	17
9 Test Specifications	18
10 Test Results	33
11 Regression Testing	41
IV Inspection.....	43
12 Items to be Inspected.....	44
13 Inspection Procedures	44
14 Inspection Results	46
V Recommendations and Conclusions	51
VI Project Issues.....	52
16 Waiting Room	53
17 Ideas for Solutions	54
18 Project Retrospective	56
VII Glossary	57
VIII References / Bibliography	60
IX Index	61

List of Figures

<i>Figure 1 – Dragon’s Lair screen for Greedy Dwarf Game.....</i>	<i>8</i>
<i>Figure 2 – Inventory Screen for Greedy Dwarf Game.....</i>	<i>9</i>
<i>Figure 3 – Character Selection for Greedy Dwarf Game.....</i>	<i>13</i>

List of Tables

<i>Table 1 -Test Results Summary.....</i>	<i>37</i>
---	-----------

I Project Description

1 Project Overview

Greedy Dwarf is a captivating top-down arcade-style game inspired by classic titles like Pac-Man. The game is set within a richly designed fantasy dungeon. Players control a dwarf whose mission is to collect valuable treasures guarded by a dragon. Navigating through intricately laid-out dungeons, players must strategically gather as many treasures as possible to maximize their score while evading the dragon's fiery retaliation. As the dwarf accumulates more treasures, movement speed decreases, introducing a layer of strategic inventory management. The game is designed for quick, engaging play sessions, making it ideal for both casual and dedicated gamers. The key issue behind the idea takes inspiration from the popular Knapsack Problem, where someone must maximize the amount of value in items stored in a knapsack while minimizing the cost.

2 Project Domain

Greedy Dwarf operates within the arcade-action and adventure genres, blending fast-paced treasure collection with strategic evasion mechanics. The game's core elements include:

- **Unique Points of Interests (POI):** Each Point of Interest within Greedy Dwarf offers a fresh and engaging experience, featuring distinct environments and challenges such as Jungle City, Dragon Lair, and Crossroads City. These POIs are meticulously designed to provide varied gameplay dynamics, encouraging exploration and rewarding players with unique treasures and interactive opportunities.
- **Inventory Management:** Effective inventory management is a critical component of Greedy Dwarf. Players must balance the weight of collected treasures against the dwarf's mobility. As the inventory accumulates more treasures, the dwarf's movement speed decreases, introducing a strategic layer where players must decide which treasures to prioritize and which to potentially leave behind. This mechanic adds depth to the gameplay, compelling players to make thoughtful decisions that impact their overall performance and progression.
- **Intelligent Enemy AI:** The dragon antagonist in Greedy Dwarf employs a pathfinding algorithm, specifically A*, to pursue the dwarf throughout the dungeon. This intelligent AI adapts to the player's movements and the dungeon's layout, ensuring a challenging and dynamic adversary that requires constant vigilance and strategic planning. The dragon's adaptive behavior heightens the tension and urgency, making each encounter uniquely challenging.

- **Interactive Elements:** Greedy Dwarf features a variety of interactable tiles and objects that enrich the gameplay experience. These include doors that require specific keys to unlock, treasure chests containing valuable items, and activatable items that grant temporary advantages or defensive capabilities. These interactive elements encourage players to engage strategically with their environment, fostering a sense of exploration and discovery as they navigate through the dungeon's complexities.
- **Character Diversity:** Players can choose from three distinct dwarf characters, each with unique attributes that influence gameplay dynamics:
 - Scout Dwarf: Excels in movement speed, allowing for rapid navigation through dungeons. However, this comes at the cost of reduced carrying capacity, limiting the amount of treasure that can be collected at any given time.
 - Miner Dwarf: Boasts a higher carrying capacity, enabling the collection of more treasures. This advantage is balanced by a slower movement speed, requiring players to adopt a more deliberate and strategic approach to navigating the dungeon.
 - Warrior Dwarf: Offers a middle ground with moderate movement speed and carrying capacity. This versatility allows players to experience a balanced gameplay style, combining both speed and the ability to collect a reasonable number of treasures.

3 Relationship to Other Documents

Project Description: The foundation of our project was built on the layout of the Project Description Document. It provided an overview of the game's concept, purpose, and inspiration from classic arcade titles like Pac-Man. The document also provides a reference point for the game's scope, audience, and mechanics of the product. One of these mechanics was the Knapsack Problem, which we integrated into the gameplay. The Project Description Document guides our design and features for the game.

Project Requirements: Building on the initial description of the Project Requirements Document, helped us organize the functional and non-functional requirements necessary for the Greedy Dwarf game. It outlined things such as interface options, control schemes, and overall flow of the game. This gave the team the ability to organize tasks effectively and have a clear weekly goal for the project.

Product Use Cases: The document provided detailed scenarios on how a user would interact with the game. It included diagrams, lists, and descriptions that helped organize the development of the product. The use cases guided the team's design of the controls, gameplay mechanics, and main navigation of the game.

Design: The Design Document helped the decomposition of the Greedy Dwarf project. It helped set the system architecture, class design, and the different subsystems in the product. These subsystems include components such as controller, creature behavior, and pickup systems. Overall, the design document was essential during the coding phase of the project. It ensured that each object and class functioned correctly.

4 Naming Conventions and Definitions

4a Definitions of Key Terms

Content:

- (1) User, Player** - The person interacting with the program
- (2) Program, Game, Project** - The game Greedy Dwarf
- (3) App Store, Google Play store, Steam** - the service from which the game will be downloaded
- (4) Valve** - the development company which creates video games
- (5) Windows, Apple iOS, Linux** - the computer system environment on which the game will be installed
- (6) Weight, value** – The game mechanics that the dwarf are mainly concerned with, as the player picks up items, the dwarf accumulates a different weight and value, and has to choose between the tradeoffs of both, all while avoiding the dragon.
- (7) Game Characters & Important Features:** the features present in the game, such as character types, speed, weight, values, POIs, and treasure types.
- (8) Dwarf** - the playable character in Greedy Dwarf which the player commands and uses to interact with the game world.
- (9) Dragon** - the main NPC antagonist in the game who attempts to find and defeat the player.
- (10) Dragon's Lair** - the location where the game takes place, where the dragon lives.
- (11) Treasure** - what the player, as the dwarf, collects from the dragon's lair. Includes gold, which weighs little but is also worth little, rarer treasures which weigh much and are worth a lot, and items from chests which may not affect the score but have incredible functionality to the player
- (12) POI** – The enterable areas for the player/dwarf that unveil new treasures and obstacles for the player.
- (13) Key** – the item that can be picked up that unlocks doors to reveal other treasures

(14) Scout, Warrior, Miner – The character types associated with a dwarf, the player can choose one of three to play as, all containing different characteristics involving speed and carrying capacity.

(15) Exit – The escape mechanism to which the player has won the game

(16) Pause – The button the player can press to pause the game or quit

(17) Leaderboard – The associated score-keeping mechanism to keep the player informed about the top scores they have accumulated locally as well as globally

(18) Instructions – The interface that displays instructions on how to navigate through the game, from movement to escape, including the associated weights/values of each treasure.

4b UML and Other Notation Used in This Document

This document generally follows the UML standard found in “UML Distilled” by Fowler et. all.

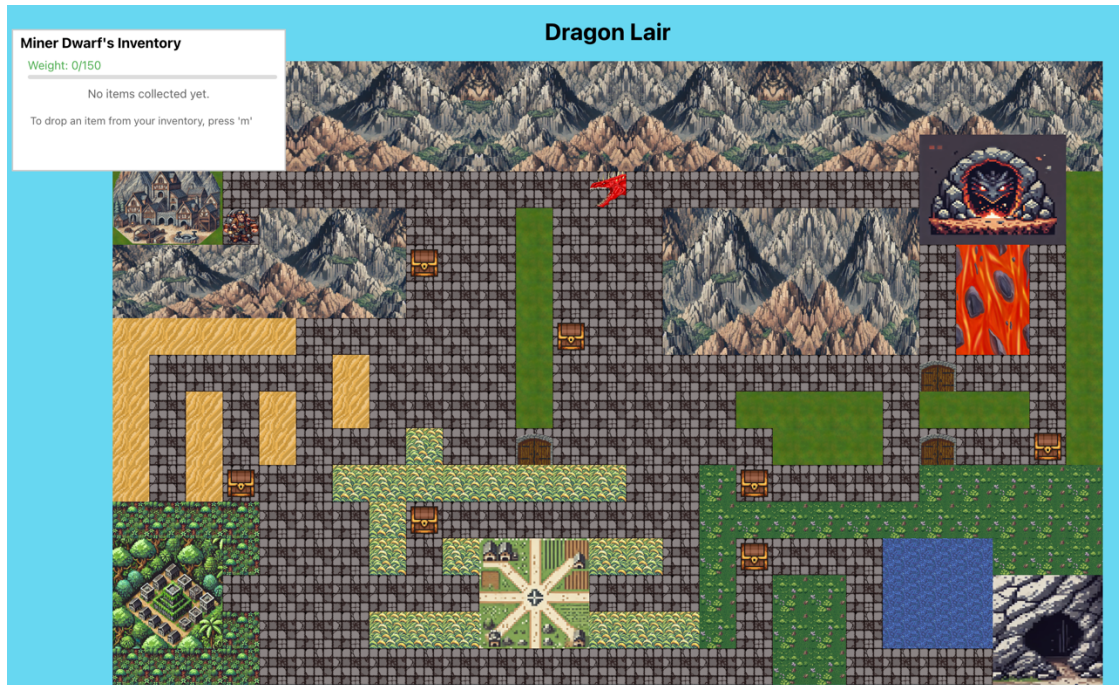
4c Data Dictionary for Any Included Models

Map Data

Map Matrix = two-dimensional array of characters representing game tiles

- Dimensions: NUM_ROWS x NUM_COLS (17 x 27)
- Valid tile types:
 - "P" = Path (walkable)
 - "W" = Wall (non-walkable)
 - "D" = Dwarf position
 - "T" = Treasure chest
 - "B[1-9]" = Door with ID number
 - "K[1-9]" = Key with ID number
 - "E" = Exit
 - "A" = Dragon Lair entrance
 - "X" = Jungle City entrance
 - "C" = Crossroads City entrance

Dragon's Lair screen for Greedy Dwarf Game



Character Statistics

Character Stats Record = {name + image + movement speed + carry capacity + special abilities}

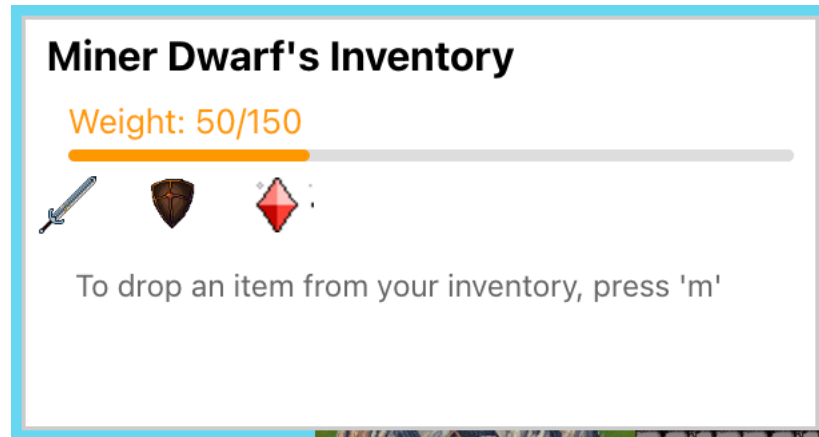
- Name = String identifying character type ("Warrior Dwarf", "Scout Dwarf", "Miner Dwarf")
- Movement Speed = Base movement delay in milliseconds (varies by character)
- Carry Capacity = Maximum weight threshold in numeric units
- Character Description = Text describing character's strengths

Inventory System

Inventory = Array of collected items

- Item Types: Treasures and Keys
- Maximum capacity determined by character's weight threshold
- Weight Thresholds = {medium threshold + heavy threshold + maximum threshold}

Inventory Screen for Greedy Dwarf Game



Treasure System

Treasure Definition = {treasure name + weight value + score value}

- Weight values range from 1-10 units
- Score values range from 10-100 points

Randomized Treasures Map = {map name + position coordinates + treasure type}

- Tracks treasure placement and collection status
- Position coordinates stored as "mapName-row-col"

Map State Management

Map States = {map name + current map matrix + collected treasures + door states}

- Tracks state of each distinct map area
- Persists changes when moving between areas
- Includes main map and all Points of Interest (POI)

Dragon AI System

Dragon Position = {row coordinate + column coordinate}

- Updated every movement interval
- Separate position tracked for each map

Movement Pattern = A* pathfinding algorithm output

- Calculates optimal path to dwarf
- Updates every second unless paused

Game Progress Data

Game Result Record = {username + score + game outcome + timestamp}

- Username = Player identifier string
- Score = Numeric point total
- Outcome = "WON" or "LOST"
- Stored in external database for leaderboard tracking

This data dictionary defines the core data structures that facilitate the game's mechanics and state management. Each structure has been designed to support specific gameplay elements while maintaining clean separation of concerns.

II Project Deliverables

5 First Release

Early Exploration Gameplay Development

In our initial release phase, we focused on establishing fundamental gameplay systems and player interactions within the Dragon's Lair environment.

Core Gameplay Implementation

We developed the primary gameplay loop centered around treasure collection and strategic movement. Players control a dwarf character who must navigate through the procedurally loaded Dragon's Lair environment. The game utilizes a flexible map file system that enables dynamic environment generation and customization.

Movement and Control Systems

We implemented player controls using both WASD and arrow key configurations, enabling fluid movement in four directions, similar to a game like Pac-Man, whom this game took inspiration from. The control scheme includes essential game management functions, with the escape key or 'q' for game termination, and space bar or 'p' for accessing the pause menu functionality.

Inventory and Weight Management

A sophisticated inventory system was developed to track collected treasures and their associated attributes. The system monitors both item value and weight, displaying this information prominently in the upper right corner of the interface. We implemented a dynamic movement speed system that adjusts based on the player's current cargo weight, with carrying capacity thresholds at:

Low Load (0-39%): Indicated by green UI elements Medium Load (40-79%): Marked with orange indicators Heavy Load (80%+): Displayed in red, triggering warning notifications.

When players exceed 100% capacity, we froze the dwarf and forced the player to choose an item to drop before allowing the dwarf to move further, which kept realistic ties to the original concept of the game.

Key and Door Mechanics

We developed key and door systems featuring unique key-door pairs. The implementation includes:

- A notification system that displays key acquisition messages
- Automatic door interaction when holding corresponding keys
- Weight adjustment calculations upon key usage
- An inventory interface allowing players to identify specific keys through tooltips

User Interface Development

Our UI implementation includes:

- A comprehensive main menu system
- In-game status displays for inventory management
- Contextual pop-up windows for player interactions
- Color-coded weight indicators
- Clear messaging for player feedback and guidance

Interaction System

We implemented a streamlined interaction system utilizing the 'M' key as the primary interaction mechanism. This system manages:

- Treasure collection
- Key acquisition
- Door manipulation
- Contextual feedback through pop-up notifications

Dragon Variable

The first release of the game includes a naïve dragon, with minor pathfinding capabilities, named Smart Search. Smart search simply finds the Manhattan Distance between the dragon AI and the dwarf and moves accordingly to decrease the distance. At times, it is not the best move, and oftentimes it caused issues in the implementation. The first release featured a dragon only partially working but was fixed in the subsequent release(s).

Success Conditions

The completion criteria for this phase includes successful treasure collection and navigation through the environment. Players must effectively manage their inventory weight while collecting valuable items and locating necessary keys for progression.

This initial release establishes the foundational systems required for the core gameplay loop, setting the stage for future expansions and additional feature implementations.

6 Second Release

Following our successful first phase, we have completed a comprehensive expansion of the Greedy Dwarf game, implementing sophisticated gameplay mechanics and enhanced user engagement features. This report details the substantial improvements and additions made during our second and final release phase.

User Authentication and Progression Systems

We implemented a robust user authentication system that tracks individual player progress and achievements. Players now begin each session by entering their username, which the system uses to maintain persistent profiles. This addition enables personalized gameplay experiences and long-term engagement tracking.

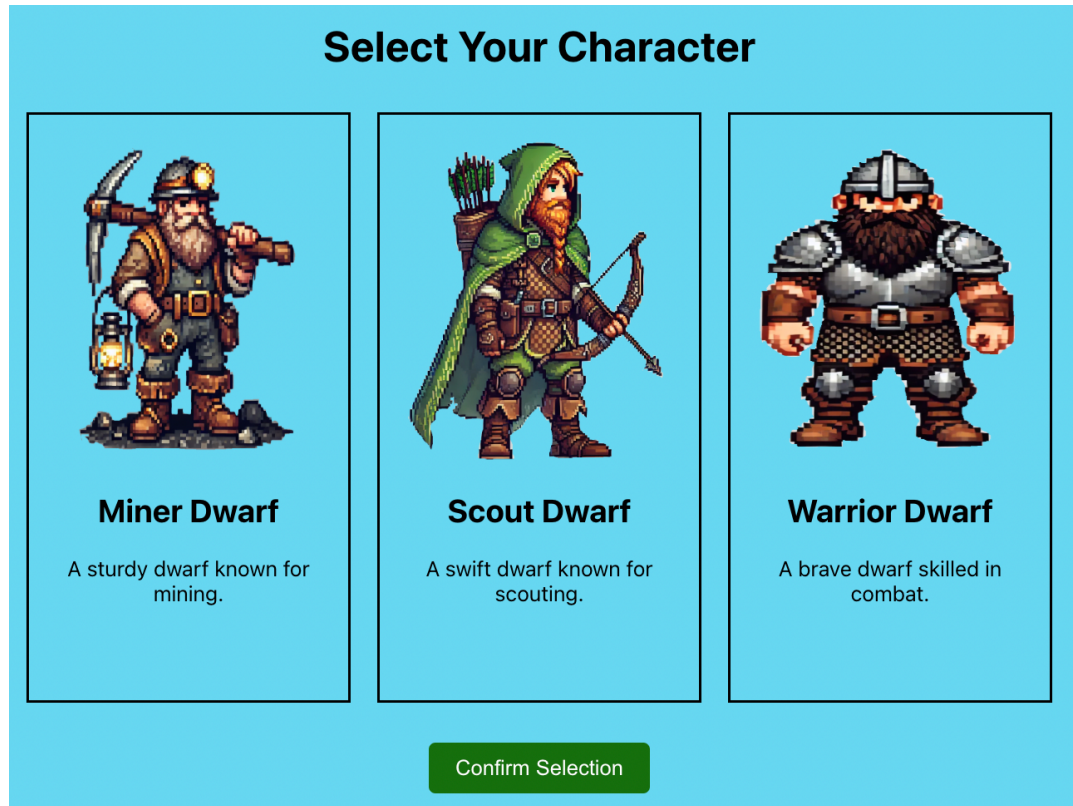
Character Selection Framework

A new character selection system provides players with meaningful gameplay variations through three distinct dwarf classes:

- The Miner Dwarf specializes in high carrying capacity at the cost of movement speed
- The Scout Dwarf excels in mobility but has limited carrying capacity
- The Warrior Dwarf offers balanced attributes for versatile gameplay

The system remembers player preferences and offers a confirmation screen for returning users, streamlining the gameplay experience while maintaining player choice.

Character Selection Screen for Greedy Dwarf Game



Points of Interest (POI) System

We developed an extensive POI framework that significantly expands the game world. Players can now explore three distinct locations beyond the main map:

- Jungle City
- Dragon's Cave
- Crossroads City

Each POI functions as a fully realized environment with its own layout, treasure distribution, and strategic considerations. Players transition between areas using an intuitive "E" key interaction system, with clear entrance and exit mechanics preserving gameplay continuity.

Advanced Dragon AI Implementation

We significantly enhanced the dragon's pursuit mechanics by implementing A* pathfinding algorithm, replacing the previous Smart Search system. This upgrade provides more sophisticated and efficient enemy behavior, creating:

- Optimal pathfinding through complex environments
- More challenging and realistic pursuit patterns
- Better navigation around obstacles and through POIs
- Improved performance in calculating pursuit routes

The A* implementation allows the dragon to effectively track the player across all environments, including within POIs, making escape and evasion more strategic and engaging.

Leaderboard Integration

We implemented a comprehensive scoring system with both global and personal leaderboards. This feature:

- Tracks and displays all-time high scores across the player base
- Maintains detailed personal performance histories
- Records specific achievement dates and character selections
- Enables score comparison and competitive play

Expanded Inventory Management

The inventory system received substantial upgrades, including:

- Weight-based movement penalties specific to character classes
- Visual feedback through color-coded weight indicators
- Improved item management with selective dropping capabilities
- Persistent inventory tracking across different map areas

Technical Achievements

The implementation leverages React's advanced features for optimal performance:

- Efficient state management using hooks and callbacks
- Real-time game state updates across multiple environments
- Seamless transitions between different game areas (and through callbacks, allows the dwarf to pick up where they left off, exactly where they entered a POI, with the same conditions as before they entered).

- Responsive user interface elements adapting to game states

Map State Management

We developed a sophisticated map state management system that:

- Preserves environment states across transitions
- Maintains consistent treasure and key placement
- Tracks collected items across all areas
- Ensures proper game progression through multiple environments

This release significantly expands upon our initial implementation, creating a more engaging and technically sophisticated gaming experience while maintaining the core treasure-hunting mechanics that defined our first release.

7 Comparison with Original Project Design Document

Overview Changes

The original project by Aaron Lau, Christian Rivera, and Matthew Searcy laid out a foundation for a Pac-Man-inspired treasure collection game, which our implementation has evolved while maintaining the core concept. The most significant changes occurred in gameplay mechanics, technical implementation, and feature scope.

Key Architectural Differences

Dragon AI Implementation

- Original Design: Proposed a simple Manhattan Distance-based "Smart Search" pathfinding
- Our Implementation: Enhanced to A* pathfinding algorithm, providing more sophisticated enemy behavior and improved navigation through complex environments

Map Management

- Original Design: Single environment with procedural loading
- Our Implementation: Expanded to include multiple Points of Interest (POI):
 - Jungle City
 - Dragon's Cave
 - Crossroads City
 - Implementation of seamless transitions between areas
 - State preservation across environment transitions

Character System

- Original Design: Single dwarf character type
- Our Implementation: Introduced three distinct character classes:
 - Miner Dwarf (high capacity, low speed)
 - Scout Dwarf (high mobility, low capacity)
 - Warrior Dwarf (balanced attributes)

Feature Evolution

User Authentication

- Original Design: Basic save system
- Our Implementation: Added:
 - Robust user authentication
 - Personal profiles
 - Progress tracking
 - Achievement system

Inventory System

- Original Design: Basic weight-based system with three thresholds
- Our Implementation: Enhanced with:
 - Character class-specific weight penalties
 - Improved visual feedback
 - Selective item dropping capabilities
 - Cross-map inventory persistence

Technical Implementation

- Original Design: Unity-based implementation
- Our Implementation: React-based with:
 - Advanced state management using hooks
 - Real-time updates
 - Cross-environment state persistence
 - Responsive UI elements

Retained Core Elements

Several key features from the original design were maintained:

1. Weight-based movement system
2. Key-door mechanics
3. Core treasure collection gameplay
4. Dragon pursuit mechanics

5. Basic UI elements for inventory and status display

Scope Adjustments

The original design's scope was expanded in our implementation while maintaining its core gameplay loop. The addition of multiple environments, character classes, and enhanced AI demonstrates significant evolution from the original concept while preserving its fundamental appeal.

Technical Infrastructure

The shift from Unity to React represents the most substantial difference from the original design, changing how the game is delivered and maintained. This change enables better web integration and cross-platform compatibility without compromising the original design's gameplay vision.

Conclusion

While maintaining the spirit of the original design document, our implementation has expanded upon its foundation with more sophisticated systems, enhanced gameplay mechanics, and improved infrastructure. The changes reflect both modern gaming trends and technical capabilities while preserving the core gameplay loop that made the original concept compelling.

III Testing

8 Items to be Tested

- **IT1: Dwarf Movement Mechanics**

The dwarf moves correctly in response to player input.

- **IT2: Dragon AI Pursuit Mechanics**

The dragon follows and tries to catch the dwarf using pathfinding algorithms.

- **IT3: Treasure Interaction Mechanics**

The dwarf can pick up and drop treasures.

- **IT4: Score Display Functionality**

The game displays the correct score based on collected treasures.

- **IT5: Map Transition Mechanics**

The player can transition between maps by stepping into designated areas and return to previous map.

- **IT6: Leaderboard Display Functionality**

Leaderboards display the correct scores and player information.

- **IT7: Leaderboard Search Feature**

The leaderboard includes a functional search capability.

- **IT8: Win and Lose Conditions**

The game recognizes when the dwarf escapes or is captured by the dragon and responds appropriately.

- **IT9: Movement Constraints**

The dwarf and dragon can only move through designated paths on the map.

- **IT10: Character Class Attributes**

Different dwarf classes (Scout, Miner, Warrior) have unique attributes affecting gameplay.

- **IT11: Inventory Management Mechanics**

The dwarf's movement speed decreases as more treasure are collected.

- **IT12: Interactive Elements Functionality**

Door, key and chest items function correctly.

- **IT13: User Interface Consistency**

UI displays correctly.

- **IT14: Performance Standards**

The game maintains consistent performance (e.g., frame rate) across different devices.

9 Test Specifications

Test ID: T1 - Dwarf Movement Test

Description: Verify that the dwarf moves correctly in response to player input.

Items Covered by this Test:

- IT1: Dwarf Movement Mechanics

Requirements Addressed by this Test:

- Requirement 10.5 (Controls 1)
- Requirement 10.6 (Controls 2)
- Requirement 12.2 (Response Time)

Environmental Needs:

- Testing device (PC, iOS, and web browser).
- Input device (keyboard/mouse).

Intercase Dependencies:

- N/A

Test Procedures:

1. Launch the game and start a new game session.
2. Use input controls to move the dwarf in all possible directions (up, down, left, right).
3. Observe the dwarf's movement on the screen.
4. Repeat the test on different platforms (PC, iOS, Android).

Input Specification:

Directional inputs via keyboard keys (e.g., W/A/S/D)

Output Specifications:

1. The dwarf moves one unit in the corresponding direction of the input.
2. Movement occurs within 100ms of input (no noticeable delay).

Pass/Fail Criteria:

- Pass: The dwarf moves correctly in the direction of the input with immediate response.
- Fail: The dwarf does not move, moves in the wrong direction, or there is a significant delay.

Test ID: T2 - Dragon Pursuit Test

Description: Verify that the dragon follows and tries to catch the dwarf using the pathfinding algorithm.

Items Covered by this Test:

- IT2: Dragon AI Pursuit Mechanics
- IT13: Dragon Pathfinding Algorithm

Requirements Addressed by this Test:

- Requirement 10.4 (Levels, Content, and Gameplay)
- Requirement 15.5 (Feedback)

Environmental Needs:

- Game running with levels where the dragon is present.

Intercase Dependencies:

- T1 must pass to ensure dwarf movement is functioning.

Test Procedures:

1. Game running with levels where the dragon is present.
2. Move the dwarf around the map.
 - a. Infront of the dragon.
 - b. Behind walls with respect to the dragon.
 - c. Behind treasure chests with respect to the dragon.
 - d. Behind locked doors with respect to the dragon.
3. Observer the dragon movement.
4. Make sure the dragon follows the dwarf and avoids obstacles.
5. Repeat test in different map environments.

Input Specification:

- Player movement inputs to collect treasure and navigate the map.

Output Specifications:

- The dragon adjusts its path to pursue the dwarf efficiently.
- The dragon avoids obstacles and takes the shortest possible path.

Pass/Fail Criteria:

- Pass: The dragon effectively pursues the dwarf using the shortest paths, adapting to the dwarf's movements.
- Fail: The dragon fails to pursue, gets stuck, or does not adjust its path appropriately.

Test ID: T3 - Treasure Interaction Test

Description: Verify that the dwarf can pick up and drop treasures and inventory updates accordingly.

Items Covered by this Test:

- IT3: Treasure Interaction Mechanics
- IT11: Inventory Management Mechanics

Requirements Addressed by this Test:

- Requirement 10.4 (Levels, Content, and Gameplay)
- Requirement 15.1 (Efficiency of Use)

Environmental Needs:

- Game running with accessible treasures.
- Interface to view inventory (if applicable).

Intercase Dependencies:

- T1 must pass.

Test Procedures:

1. Start a game session with the dwarf near a treasure.
2. Move the dwarf over the treasure to pick it up.
3. Check if the treasure disappears from the map and appears in the inventory.
4. Attempt to drop the treasure using the appropriate command.
5. Verify the treasure reappears on the map at the dwarf's location.
6. Observe any changes in the dwarf's movement speed due to inventory weight.

Input Specification:

- Movement inputs to pick up treasures.
- Command inputs to drop treasures.

Output Specifications:

- Treasure is removed from the map when picked up.
- Treasure is added to the inventory.
- Dwarf's movement speed adjusts according to inventory weight.
- Treasure reappears on the map when dropped.

Pass/Fail Criteria:

- Pass: The dwarf can pick up and drop treasures successfully, with inventory and movement speed updating correctly.
- Fail: Treasure pickup/drop does not function, inventory does not update, or movement speed does not adjust.

Test ID: T4 - Score Display Test

Description: Verify that the game displays the correct score based on collected treasures.

Items Covered by this Test:

- IT4: Score Display Functionality

Requirements Addressed by this Test:

- Requirement 10.3 (Game Statistics)
- Requirement 16.3 (Font and Logos)

Environmental Needs:

- Game running.
- Access to treasures with known values.

Intercase Dependencies:

- T3 must pass.

Test Procedures:

1. Start a new game session.
2. Note the initial score (should be zero).
3. Collect a treasure worth a specific value.
4. Check if the score updates to reflect the treasure's value.
5. Collect additional treasures and verify cumulative score.
6. Drop a treasure and verify if the score adjusts accordingly (if game design dictates).

Input Specification:

- Movement inputs to collect and drop treasures.

Output Specifications:

- Score display updates immediately after collecting or dropping treasures.
- Score reflects the total value of treasures currently held.

Pass/Fail Criteria:

- Pass: Score accurately reflects the value of collected treasures and updates in real-time.
- Fail: Score does not update, updates incorrectly, or displays inconsistently.

Test ID: T5 - Map Transition Test

Description: Verify that the player can transition between maps by stepping into designated areas and return.

Items Covered by this Test:

- IT5: Map Transition Mechanics

Requirements Addressed by this Test:

- Requirement 10.4 (Levels, Content, and Gameplay)

Environmental Needs:

- Game running with multiple interconnected maps.
- Designated transition areas available.

Intercase Dependencies:

- T1 must pass.

Test Procedures:

1. Start a game session and navigate to a map transition area.
2. Step into the area and observe if the new map loads correctly.
3. Verify the dwarf's position in the new map is appropriate.
4. Attempt to return to the original map by re-entering the transition area.
5. Repeat the process for different maps and transition points.

Input Specification:

- Movement inputs to reach and enter transition areas.

Output Specifications:

- Smooth transition between maps without loading errors.
- Dwarf retains inventory and game state across maps.
- Return to previous maps functions correctly.

Pass/Fail Criteria:

- Pass: Map transitions occur seamlessly, and the game state persists.
- Fail: Transitions cause errors, crashes, or loss of game state.

Test ID: T6 - Leaderboard Display Test

Description: Verify that the leaderboards display the correct values and players.

Items Covered by this Test:

- IT6: Leaderboard Display Functionality

Requirements Addressed by this Test:

- Requirement 10.3 (Game Statistics)
- Requirement 14.2 (Supported Platforms)

Environmental Needs:

- Game connected to the server (if leaderboards are online).
- Multiple player scores recorded.

Intercase Dependencies:

- T4 must pass to ensure scores are recorded correctly.

Test Procedures:

1. Finish a game session with a high score.
2. Access the leaderboards from the main menu.
3. Verify that the new score appears in the correct position.
4. Compare displayed scores with known data from other test accounts.
5. Check for consistency across different platforms.

Input Specification:

- Completion of game sessions with varying scores.

Output Specifications:

- Leaderboard displays player names and scores accurately.
- Rankings are correctly ordered from highest to lowest scores.

Pass/Fail Criteria:

- Pass: Leaderboards display accurate and updated information for all players.
- Fail: Scores are missing, incorrect, or rankings are not properly ordered.

Test ID: T7 - Leaderboard Search Feature Test

Description: Verify that the leaderboard's search feature functions correctly.

Items Covered by this Test:

- IT7: Leaderboard Search Feature

Requirements Addressed by this Test:

- Requirement 16.3 (Font and Logos)
- Usability and Accessibility requirements.

Environmental Needs:

- Leaderboards populated with multiple player entries.

Intercase Dependencies:

- T6 must pass.

Test Procedures:

1. Access the leaderboards.
2. Use the search feature to look for a specific player's name.
3. Verify that the search results display the correct player's score and ranking.
4. Try searching with partial names or case variations.
5. Test the search functionality on different platforms.

Input Specification:

- Player name inputs for search queries.

Output Specifications:

- Search results accurately display matching player entries.
- System handles partial matches and is case-insensitive.

Pass/Fail Criteria:

- Pass: Search feature returns correct and relevant results.
- Fail: Search results are incorrect, incomplete, or the feature does not function.

Test ID: T8 - Win/Lose Condition Test

Description: Verify that the game recognizes when the dwarf escapes or is captured by the dragon, triggering appropriate responses.

Items Covered by this Test:

- IT8: Win and Lose Conditions

Requirements Addressed by this Test:

- Requirement 15.3 (Error Rates)
- Requirement 15.5 (Feedback)

Environmental Needs:

- Game running with levels that have escape points and dragon encounters.

Intercase Dependencies:

- T1, T2, T5 must pass.

Test Procedures:

1. Play through a level and guide the dwarf to the escape point.
2. Verify that the game recognizes the win condition (e.g., displays a victory screen).
3. Restart the level and intentionally allow the dragon to catch the dwarf.
4. Verify that the game recognizes the losing condition (e.g., displays a game over screen).
5. Check that the game handles both conditions gracefully without crashes.

Input Specification:

- Movement inputs to reach escape points or encounter the dragon.

Output Specifications:

- Appropriate end-game screens or messages are displayed.
- Game records the outcome (win/loss) correctly.

Pass/Fail Criteria:

- Pass: Game correctly identifies win and lose conditions and responds appropriately.
- Fail: Game does not recognize conditions or responds incorrectly.

Test ID: T9 - Movement Constraints Test

Description: Verify that the dwarf and dragon can only travel through designated locations on the map.

Items Covered by this Test:

- IT9: Movement Constraints

Requirements Addressed by this Test:

- Requirement 15.9 (Accessibility Requirements)
- Requirement 12.4 (Maximum Level Size)

Environmental Needs:

- Game running with maps that include obstacles and restricted areas.

Intercase Dependencies:

- T1, T2 must pass.

Test Procedures:

1. Attempt to move the dwarf through walls, obstacles, or non-traversable terrain.
2. Observe whether the dwarf is prevented from moving into these areas.
3. Monitor the dragon's movement to ensure it also respects movement constraints.
4. Test various map layouts with different obstacles.

Input Specification:

- Movement inputs directed toward non-traversable areas.

Output Specifications:

- Dwarf and dragon are unable to move through walls or restricted zones.
- Collision detection prevents illegal movement.

Pass/Fail Criteria:

- Pass: Movement is constrained appropriately for both characters.
- Fail: Characters can pass through obstacles or move out of bounds.

Test ID: T10 - Character Class Attributes Test

Description: Verify that different dwarf classes have unique attributes affecting gameplay.

Items Covered by this Test:

- IT10: Character Class Attributes

Requirements Addressed by this Test:

- Requirement 15.7 (Learning Requirements)

Environmental Needs:

- Game running with access to select different dwarf classes.

Intercase Dependencies:

- T1, T3 must pass.

Test Procedures:

1. Start game sessions with each dwarf class (Scout, Miner, Warrior).
2. Measure and compare movement speeds.
3. Check carrying capacities by collecting treasures until movement speed decreases.
4. Observe any special abilities or attributes unique to each class.
5. Record the differences and verify they match design specifications.

Input Specification:

- Class selection inputs.
- Movement and treasure collection inputs.

Output Specifications:

- Scout dwarf moves faster but has lower carrying capacity.
- Miner dwarf has higher carrying capacity but moves slower.
- Warrior dwarf has balanced attributes.

Pass/Fail Criteria:

- Pass: Each class exhibits its unique attributes as intended.
- Fail: Classes do not differ, or attributes do not match specifications.

Test ID: T11 - Inventory Management Test

Description: Verify that the dwarf's movement speed decreases as more treasure is collected, reflecting the weight of the inventory.

Items Covered by this Test:

- IT11: Inventory Management Mechanics

Requirements Addressed by this Test:

- Requirement 15.1 (Efficiency of Use)
- Requirement 15.5 (Feedback)
- Requirement 15.9 (Accessibility Requirements)

Environmental Needs:

- Game installed and running.
- Access to a level with multiple treasures of varying weights.
- Stopwatch or in-game tool to measure movement speed.

Intercase Dependencies:

- T1, T3 must pass.

Test Procedures:

1. Baseline Measurement:
 - a. Start a new game session with an empty inventory.
 - b. Measure the dwarf's movement speed over a set distance (e.g., time taken to move from point A to point B).
 - c. Record the baseline speed.
2. Incremental Treasure Collection:
 - a. Collect a light treasure (e.g., gold coin) and repeat the movement speed measurement.
 - b. Record the speed.
3. Collect Additional Treasures:
 - a. Continue collecting treasures one at a time, each time measuring and recording the movement speed.
 - b. Include treasures of varying weights (light, medium, heavy).
4. Full Inventory Test:
 - a. Continue until the inventory is full or movement speed cannot decrease further.
 - b. Record the final movement speed.
5. Dropping Treasures:
 - a. Drop a treasure and measure if movement speed increases accordingly.
 - b. Repeat dropping treasures and measuring speed until back to baseline.
6. Compare Results:
 - a. Analyze the recorded speeds to see if there's a consistent decrease in speed proportional to the weight of the inventory.

Input Specification:

- Movement inputs to navigate the dwarf.
- Commands to pick up and drop treasures.

Output Specifications:

- Measurable decrease in movement speed as more treasures are collected.
- Movement speed increases when treasures are dropped.

- The change in speed should be proportional to the cumulative weight of the inventory.

Pass/Fail Criteria:

- Pass: The dwarf's movement speed decreases incrementally as treasures are collected, and increases when treasures are dropped, accurately reflecting inventory weight.
- Fail: Movement speed does not change with inventory weight, changes inconsistently, or does not adjust when treasures are dropped.

Test ID: T12 - Interactive Elements Test

Description: Verify that doors, keys, and chests items function correctly, enhancing gameplay and player interaction with the environment.

Items Covered by this Test:

- IT12: Interactive Elements Functionality

Requirements Addressed by this Test:

- Requirement 10.4 (Levels, Content, and Gameplay)
- Requirement 15.1 (Efficiency of Use)
- Requirement 15.5 (Feedback)

Environmental Needs:

- Game running with levels that include doors requiring keys, chests with items, and activatable items.
- Access to various interactive elements within the game.

Intercase Dependencies:

- T3 must pass to ensure item pickup mechanics are functioning.
- T1 must pass for movement mechanics.

Test Procedures:

1. Doors and Keys:
 - a. Locate a locked door in the game.
 - b. Attempt to open it without a key; verify that it remains locked.
 - c. Find the corresponding key.
 - d. Use the key to unlock the door.
 - e. Verify the door opens and allows passage.

2. Chests:

- a. Approach a chest and interact with it.
- b. Verify that the chest opens and reveals its contents.
- c. Collect the items from the chest.
- d. Ensure the chest cannot be reopened once emptied (if design specifies).

Input Specification:

- Interaction inputs to use keys, open chests, and activate items.
- Movement inputs to navigate to interactive elements.

Output Specifications:

- Doors unlock and open when the correct key is used.
- Chests open and display contents when interacted with.
- Activatable items produce the intended effects on the dwarf or environment.
- Visual and audio feedback confirms interactions.

Pass/Fail Criteria:

- Pass: All interactive elements function as intended, with correct responses to player actions.
- Fail: Elements do not respond correctly, items do not activate, or interactions cause errors.

Test ID: T13 - User Interface Consistency Test

Description: Verify that the user interface (UI) displays correctly and consistently across all supported platforms.

Items Covered by this Test:

- IT13: User Interface Consistency

Requirements Addressed by this Test:

- Requirement 16.1 (Look and Display)
- Requirement 16.3 (Font and Logos)
- Requirement 14.2 (Supported Platforms)
- **Environmental Needs:**
 - Access to devices running PC (Windows), iOS, and Android platforms.
 - Game installed on each device.

Intercase Dependencies:

- N/A

Test Procedures:

1. Main Menu:
 - a. Launch the game on each platform.
 - b. Observe the main menu layout, fonts, logos, and options.
 - c. Ensure all elements are visible and properly aligned.
2. In-Game UI:
 - a. Start a game session.
 - b. Check the display of health bars, inventory icons, score, and other HUD elements.
 - c. Verify readability and accessibility.
3. Interactivity:
 - a. Navigate through menus and submenus.
 - b. Verify buttons and controls respond correctly to inputs.

Input Specification:

- Standard navigation inputs (mouse clicks, touch gestures, controller inputs).

Output Specifications:

- UI elements display correctly without overlapping or misalignment.
- Fonts are legible, and icons are clear.
- UI maintains a consistent look and feel across platforms.

Pass/Fail Criteria:

- Pass: The UI is consistent, functional, and visually appealing on all supported platforms.
- Fail: UI elements are misaligned, unreadable, or function inconsistently between platforms.

Test ID: T14 - Performance Standards Test

Description: Verify that the game maintains consistent performance, such as frame rate and responsiveness, across devices with varying specifications.

Items Covered by this Test:

- IT14: Performance Standards

Requirements Addressed by this Test:

- Requirement 12.3 (Frame Rate)
- Requirement 12.2 (Response Time)
- Requirement 15.1 (Efficiency of Use)

Environmental Needs:

- Devices with different hardware capabilities (high-end, mid-range, low-end).
- Tools to monitor performance metrics (e.g., FPS counters, profiling tools).

Intercase Dependencies:

- N/A

Test Procedures:

1. Run the game on High-End devices and measure the performance.
2. Run the game on Mid-Range Device and measure the performance.
3. Run the game on Low-End Device and measure the performance.
4. Stress Testing:
 - a. Simulate high-load scenarios (e.g., multiple enemies on screen, large maps).
 - b. Monitor if performance degrades significantly.

Input Specification:

- Standard gameplay inputs.
- Actions to simulate high-load scenarios.

Output Specifications:

- Game runs smoothly with consistent frame rates (e.g., 60 FPS) on recommended devices.
- Input response times meet specified requirements (e.g., actions occur within 100ms of input).
- Acceptable performance on lower-end devices with adjusted settings.

Pass/Fail Criteria:

- Pass: Game maintains required performance standards across devices, with acceptable variations.
- Fail: Significant performance issues occur, making the game unplayable or negatively affecting user experience.

10 Test Results

T1 - Dwarf Movement Test

Date(s) of Execution: 09/22/2024

Staff conducting tests: Brian Rosca

Expected Results: The dwarf moves as intended when the player uses the w, a, s, d keys on the keyboard.

Actual Results: The dwarf moved as intended.

Test Status: Pass

T2 - Dragon Pursuit Test

Date(s) of Execution: 10/04/2024, 10/20/2024, 10/11/2024

Staff conducting tests: Shaan Kohli, Victoria Rossi, Hristian Tountchev

Expected Results:

1. Test for 10/04/2024 – Dragon Simple movement – Dragon follows player most of the time.
2. Test for 10/20/2024 – Dragon Smart movement – Dragon follows player using A* algorithm. Can avoid obstacles and can get to player anywhere on the map.
3. Test for 10/11/2024 – Dragon follows players to POI (points of interest). Does so without getting stuck in obstacles.

Actual Results:

1. 10/04/2024 – The dragon followed the player as intended with issues around certain obstacles.
2. 10/20/2024 – Dragon followed the player in most instances but would get stuck. The issue was resolved in the next sprint.
3. 10/11/2024 – The dragon would follow player into POI with no issues. The A* algorithm worked in the POI as well.

Test Status: 10/04/2024 – Pass, 10/20/2024 – Fail (fixed on 10/29/2024), 10/11/2024 - Pass

T3 - Treasure Interaction Test

Date(s) of Execution: 10/22/2024, 10/28/2024, 11/07/2024, 11/15/2024

Staff conducting tests: Shaan Kohli, Hristian Tountchev

Expected Results:

1. 10/22/2024 – The player can navigate to a treasure chest and loot items from it.
2. 10/28/2024 - The player can navigate to the treasure chest and loot from a treasure chest. The chest has a random item in it. It gives the player the name, picture, score value, and weight of that item.
3. 11/07/2024 – The wight and the score of the treasures are correctly adjusted and they are free from any bugs. The look of the treasure chest and items are finalized.
4. 11/15/2024 – Special treasure can be found in different locations of the game. These locations are POI that the player can go and loot.

Actual Results:

1. 10/22/2024 – The treasure in the chest was correctly outputting the correct item.
2. 10/28/2024 – The items in the chest gave the correct name, picture, score value, and weight of that item.
3. 11/07/2024 – The item weights and scores were correctly adjusted, and they worked as intended.
4. 11/15/2024 – Treasures in chests had the correct name and score value. However, they did not have associated pictures to them and would not drop properly. The issue was fixed in the next sprint.

Test Status: 10/22/2024 – Pass, 10/28/2024 – Pass, 11/07/2024 – Pass, 11/15/2024 – Fail (fixed in a later sprint)

T4 - Score Display Test

Date(s) of Execution: 11/20/2024

Staff conducting tests: Victoria Rossi

Expected Results: The score is properly displayed on the screen for the user to see.

Actual Results: The score worked as intended and the values were correctly displayed.

Test Status: Pass

T5 - Map Transition Test

Date(s) of Execution: 11/07/2024

Staff conducting tests: Brian Rosca

Expected Results: The player can enter new maps/POIs where they can explore and loot new treasures.

Actual Results: The player was able to enter the new maps/POIs with no issues. However, there was no exit condition to go back to original map (fixed in the next sprint).

Test Status: Fail (fixed in the next sprint)

T6 - Leaderboard Display Test

Date(s) of Execution: 10/18/2024, 10/30/2024, 11/18/2024

Staff conducting tests: Brian Rosca, Victoria Rossi

Expected Results:

1. 10/18/2024 – User can create a username and play the game under that alias.
2. 10/30/2024 – User can search their username to see their scores as they play the game.
3. 11/18/2024 – Users can view all other player's scores.

Actual Results: The leader boards worked where the user could see all scores of other users. One bug found the user could not properly search their name in the leaderboards.

Test Status: 10/18/2024 – Pass, 10/30/2024 – Fail (Pass on 11/13/2024), 11/18/2024 - Pass

T7 - Leaderboard Search Feature Test

Date(s) of Execution: 10/30/2024

Staff conducting tests: Brian Rosca

Expected Results: User can search their username to see their scores as they play the game.

Actual Results: When the player executed a search of their username the database would not return the correct result.

Test Status: Fail (Pass on 11/13/2024)

T8 - Win/Lose Condition Test

Date(s) of Execution: 10/30/2024

Staff conducting tests: Hristian Tountchev

Expected Results: When a player moves the dwarf to a specific location (the big rock on the bottom right of the screen) the game ends and the winning screen is displayed. If the player is captured by the dragon, the game ends and the losing screen is displayed. On the losing and winning screen, the player is given the option to view the leader boards or go back to the main screen of the game.

Actual Results: When the player won or lost the winning and losing screens were displayed. The player had the ability to view the leader boards or return to the main screen of the game.

Test Status: Pass

T9 - Movement Constraints Test

Date(s) of Execution: 10/09/2024

Staff conducting tests: Shaan Kohli

Expected Results:

- **The user:** When the user plays the game and moves the dwarf around, they have a designated path that they can travel. Any deviation from that path is not allowed. IE walking through obstacles such as walls, treasure chests, etc.
- **The Dragon:** When the dragon follows the user, it does not bypass any obstacles and/or treasures, etc. The dragon has the same movement constraints as the user.

Actual Results: The player and the dragon moved as intended by the team's design.

Test Status: Pass

T10 - Character Class Attributes Test

Date(s) of Execution: 11/11/2024

Staff conducting tests: Victoria Rossi

Expected Results: The user before starting a game is given the option to choose from three different character types. These character types give the user a unique way to play the game. These character types are the scout, miner, and warrior.

Actual Results: The user was able to select each character type, and their abilities worked as intended.

Test Status: Fail (Pass on 11/13/2024)

T11 - Inventory Management Test

Date(s) of Execution: 10/18/2024

Staff conducting tests: Brian Rosca

Expected Results: The user can press the inventory button to open the inventory menu. In the inventory menu the user can choose which item to drop.

Actual Results: The user was able to open and drop items from their inventory menu.

Test Status: Pass

T12 - Interactive Elements Test

Date(s) of Execution: 09/22/2024, 10/07/2024, 10/10/2024

Staff conducting tests: Brian Rosca, Victoria Rossi, Shaan Kohli, Hristian Tountchev

Expected Results: The user should be able to interact with treasure chests. When looting treasure chests, the user should receive specific keys from said chests. The user should be able to use the looted keys on specific doors. No looted key should be able to open more than one door in the game.

Actual Results: The user was able to collect keys and use them on specific doors based on the unique key looted. No one key was able to open more than one door at a time.

Test Status: 09/22/2024 – Pass, 10/07/2024 – Pass, 10/10/2024 – Pass

T13 - User Interface Consistency Test

Date(s) of Execution: 10/10/2024, 10/12/2024

Staff conducting tests: Shaan Kohli, Hristian Tountchev

Expected Results: The game's UI is consistent based on documentation and across all platforms the game will be deployed on. The UI is also bug free and all attributes such as buttons work as intended.

Actual Results: The UI and its attributes were responsive and did what they were supposed to. The game's UI was built on documentation specifications.

Test Status: 10/10/2024 - Pass, 10/12/2024 - Pass

T14 - Performance Standards Test

Date(s) of Execution: 09/21/2024 – 12/06/2024

Staff conducting tests: Brian Rosca, Victoria Rossi, Shaan Kohli, Hristian Tountchev

Expected Results: The game will maintain frame rate of ~60fps and response time no more than 100ms.

Actual Results: The game's performance met the specified criteria and benchmarks. The game ran smoothly and never crashed when testing.

Test Status: 09/21/2024 – 12/06/2024 – Pass

Test Results Summary

Test ID	Test Name	Date	Staff	Expected Results Summary	Actual Results Summary	Status
T1	Dwarf Movement Test	09/22/2024	Brian Rosca	Dwarf moves with w,a,s,d keys	Moved as intended	Pass
T2	Dragon Pursuit Test	10/04/2024	Shaan Kohli	Dragon follows player with simple movement	Dragon followed with some obstacle issues	Pass
T2	Dragon Pursuit Test	10/11/2024	Hristian Tountchev	Dragon follows players to POI	Dragon followed correctly into POI	Pass
T2	Dragon Pursuit Test	10/20/2024	Victoria Rossi	Dragon follows player using A* algorithm	Dragon got stuck in some instances	Fail (fixed 10/29/2024)
T3	Treasure Interaction Test	10/22/2024	Shaan Kohli	Basic chest looting functionality	Treasure output correct	Pass
T3	Treasure Interaction Test	10/28/2024	Shaan Kohli	Random items with stats and visuals	Correct name, picture, score, weight shown	Pass

T3	Treasure Interaction Test	11/07/2024	Hristian Tountchev	Weight and score adjustment	Items worked as intended	Pass
T3	Treasure Interaction Test	11/15/2024	Hristian Tountchev	Special treasure in POI locations	Missing pictures and drop issues	Fail (fixed later)
T4	Score Display Test	11/20/2024	Victoria Rossi	Score properly displayed on screen	Worked as intended	Pass
T5	Map Transition Test	11/07/2024	Brian Rosca	Player can enter new maps/POIs	Entry worked but exit was missing	Fail (fixed next sprint)
T6	Leaderboard Display Test	10/18/2024	Brian Rosca	Username creation and alias system	Worked as intended	Pass
T6	Leaderboard Display Test	10/30/2024	Victoria Rossi	Username search functionality	Search not working properly	Fail (fixed 11/13/2024)
T6	Leaderboard Display Test	11/18/2024	Brian Rosca	View all players' scores	All scores visible	Pass
T7	Leaderboard Search Feature Test	10/30/2024	Brian Rosca	Username search functionality	Incorrect database results	Fail (fixed 11/13/2024)
T8	Win/Lose Condition Test	10/30/2024	Hristian Tountchev	Game end conditions and screens	All end conditions worked properly	Pass
T9	Movement Constraints Test	10/09/2024	Shaan Kohli	Player and dragon movement restrictions	Both entities moved within constraints	Pass
T10	Character Class	11/11/2024	Victoria Rossi	Three character	Character selection and	Fail (fixed 11/13/2024)

	Attributes Test			types with unique abilities	abilities worked	
T11	Inventory Management Test	10/18/2024	Brian Rosca	Inventory menu and item dropping	Worked as intended	Pass
T12	Interactive Elements Test	09/22/2024	Brian Rosca	Key and door interaction system	Worked correctly	Pass
T12	Interactive Elements Test	10/07/2024	Victoria Rossi	Key and door interaction system	Worked correctly	Pass
T12	Interactive Elements Test	10/10/2024	Shaan Kohli	Key and door interaction system	Worked correctly	Pass
T13	User Interface Consistency Test	10/10/2024	Shaan Kohli	UI consistency across platforms	Met specifications	Pass
T13	User Interface Consistency Test	10/12/2024	Hristian Tountchev	UI consistency across platforms	Met specifications	Pass
T14	Performance Standards Test	09/21/2024 - 12/06/2024	All Staff	60fps and <100ms response time	Met all performance benchmarks	Pass

11 Regression Testing

Throughout the development of Greedy Dwarf, several tests initially failed due to identified bugs or incomplete implementations. These tests were subsequently repeated

after the issues were addressed to confirm that the fixes were effective and that no new problems were introduced. The following outlines the regression tests conducted, their initial outcomes, the fixes applied, and the results upon retesting:

1. Dragon Pursuit Test (T2)

- **Initial Status:** Fail on 10/20/2024
- **Issue:** The dragon would occasionally get stuck while using the A* pathfinding algorithm.
- **Fix Applied:** Refined the A* implementation to better navigate around obstacles and improve pathfinding accuracy.
- **Regression Test Date:** 11/29/2024
- **Result: Pass** – The dragon consistently followed the dwarf without getting stuck, demonstrating improved pathfinding capabilities.

2. Treasure Interaction Test (T3)

- **Initial Status:** Fail on 11/15/2024
- **Issue:** Treasures lacked associated pictures and did not drop properly upon collection.
- **Fix Applied:** Updated the treasure assets to include images and refined the drop mechanics to ensure treasures are correctly removed from the map and added to the inventory.
- **Regression Test Date:** 11/20/2024
- **Result: Pass** – Treasures now display appropriate images and interact with the inventory system as designed, enhancing the visual and functional aspects of treasure collection.

3. Map Transition Test (T5)

- **Initial Status:** Fail on 11/07/2024
- **Issue:** There was no exit condition to return to the original map after entering a new POI.
- **Fix Applied:** Implemented exit points within each POI to allow players to seamlessly return to previous maps.
- **Regression Test Date:** 11/15/2024
- **Result: Pass** – Players can now transition between maps and return to the original map without issues, ensuring consistent navigation across all environments.

4. Leaderboard Display Test (T6)

- **Initial Status:** Fail on 10/30/2024
- **Issue:** Users could not properly search their names in the leaderboards.

- **Fix Applied:** Corrected the search query logic to accurately filter and retrieve player scores based on input.
- **Regression Test Date:** 11/18/2024
- **Result: Pass** – The leaderboard search feature now correctly retrieves and displays the appropriate player scores, handling partial and case-insensitive inputs effectively.

5. Leaderboard Search Feature Test (T7)

- **Initial Status:** Fail on 10/30/2024
- **Issue:** The leaderboard search feature did not return correct results when searching for usernames.
- **Fix Applied:** Corrected the search query logic to accurately filter and retrieve player scores based on input.
- **Regression Test Date:** 11/13/2024
- **Result: Pass** – The search functionality now correctly retrieves and displays the appropriate player scores, handling partial and case-insensitive inputs effectively.

6. Character Class Attributes Test (T10)

- **Initial Status:** Fail on 11/11/2024
- **Issue:** Character classes did not exhibit their unique attributes as intended.
- **Fix Applied:** Adjusted the character class configurations to ensure each class (Scout, Miner, Warrior) had distinct movement speeds and carrying capacities.
- **Regression Test Date:** 11/13/2024
- **Result: Pass** – Each character class now correctly displays its unique attributes, providing the intended gameplay variations and strategic depth.

To maintain the quality and stability of Greedy Dwarf, regression testing should be conducted under the following conditions:

- **After Bug Fixes:** Any time a bug is identified and fixed, relevant tests should be re-executed to ensure the fix resolves the issue without introducing new problems.
- **Upon Feature Enhancements:** When new features are added or existing features are enhanced, regression tests related to those areas should be performed to verify that integrations are seamless and that existing functionalities remain unaffected.
- **Periodic Testing:** Regular regression testing cycles should be scheduled, especially before major releases, to ensure ongoing stability and performance across all game components.

IV Inspection

This portion of the report documents the inspection procedures applied, and the results obtained. In our CS 440 project, each team member contributed a significant piece of code to

be inspected by the remainder of the group. Our team consists of four members: **Brian Rosca**, **Victoria Rossi**, **Shaan Kohli**, and **Hristian Tountchev**. Each member submitted a code module for inspection by the other three team members.

12 Items to be Inspected

In accordance with the inspection procedures, the following items were selected for inspection:

1. Inventory Management System

- **Contributor:** Hristian Tountchev
- **Description:** This module handles the collection, storage, and management of treasures and keys within the player's inventory. It includes weight calculations, item stacking, and interaction mechanics for picking up and dropping items.

2. Dragon AI Pathfinding Algorithm

- **Contributor:** Brian Rosca
- **Description:** This component implements the A* pathfinding algorithm for the dragon's pursuit of the dwarf. It ensures intelligent navigation around obstacles and efficient tracking of the player's movements across different maps and POIs.

3. Map Transition and State Management

- **Contributor:** Shaan Kohli
- **Description:** This code manages transitions between different maps and Points of Interest (POIs), maintaining consistent game states, environment data, and player positioning across transitions.

4. User Interface (UI) Development

- **Contributor:** Victoria Rossi
- **Description:** This section includes the implementation of the game's UI elements, such as menus, in-game displays, and interaction notifications.

13 Inspection Procedures

To ensure code quality and reliability, we conducted formal code inspections following a structured procedure:

Preparation

- **Submission:** Each team member submitted their code module one week prior to the scheduled inspection meeting.
- **Distribution:** Code and relevant documentation were distributed to the inspectors (the other team members) via a shared Google Docs.
- **Familiarization:** Inspectors reviewed the code individually, familiarizing themselves with the functionality and noting any preliminary observations.

Inspection Checklist

We utilized a customized code inspection checklist adapted from the **IEEE Standard for Software Reviews and Audits (IEEE Std 1028-1997)**. The checklist focused on:

- **Functionality:**
 - Does the code meet the specified requirements?
 - Are all features implemented as intended?
- **Correctness:**
 - Is the logic error-free?
 - Are calculations and data manipulations accurate?
- **Coding Standards:**
 - Does the code adhere to the team's coding conventions?
 - Are naming conventions and formatting consistent?
- **Readability and Maintainability:**
 - Is the code well-organized and modular?
 - Are comments and documentation sufficient?
- **Performance:**
 - Is the code optimized for efficiency?
 - Are there any bottlenecks or unnecessary resource usages?
- **Security and Error Handling:**
 - Are potential exceptions and errors properly handled?

- Is user input validated to prevent security vulnerabilities?

Inspection Meetings

- **Meeting Schedule:**

- **First Meeting:** November 20, 2024 (2 hours, in person at the CS Lounge)
- **Second Meeting:** December 2, 2024 (2 hours, in person at the CS Lounge)

- **Agenda:**

- **First Meeting:** Each contributor presented their code, and initial observations were discussed.
- **Between Meetings:** Inspectors conducted in-depth reviews individually, documenting issues.
- **Second Meeting:** Detailed findings were shared, and action items were assigned.

Communication of Results

- **Documentation:** Inspection findings were recorded in a shared Google Doc, categorized by severity (Critical, Major, Minor).
- **Discussion:** Issues were discussed collaboratively to understand their implications.
- **Assignment:** Contributors were assigned tasks to address the identified issues.

Follow-up and Resolution

- **Corrections:** Contributors made the necessary code changes.
- **Verification:** Inspectors reviewed the updated code to ensure issues were resolved.
- **Re-inspection:** If necessary, code was re-inspected to confirm the effectiveness of fixes.

14 Inspection Results

For each inspection that was performed, we documented what was inspected, who did the inspection, the time and date when the inspection was performed, and the findings from the inspection. Notably, we highlighted any flaws that were revealed, the resolutions applied, and whether any re-inspections occurred.

1. Inventory Management System

- **Contributor:** Hristian Tountchev
- **Inspectors:** Brian Rosca, Victoria Rossi, Shaan Kohli
- **Inspection Dates:** November 20, 2024, and December 2, 2024

Findings:

- **Issue 1: Duplicate Item Removal Bug**
 - **Severity:** Major
 - **Description:** When a player attempted to remove an item that had multiple instances in the inventory (e.g., two or more of the same item), all instances of that item were removed, but the inventory weight was only adjusted for one instance.
 - **Resolution:** Hristian modified the inventory management system to accurately track the quantity of each item. He implemented a stack-based approach where each inventory item includes a count of how many instances are held. The code was adjusted to decrement the count appropriately and update the inventory weight accordingly when an item is removed.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors verified that the issue was resolved; removing one instance of an item now only reduces the count by one and adjusts the weight correctly.
- **Issue 2: Incorrect Weight Calculations after Dropping Items**
 - **Severity:** Minor
 - **Description:** When items were dropped from the inventory, the total weight was not recalculated correctly, leading to inaccuracies in movement speed adjustments.
 - **Resolution:** Hristian added a function to recalculate the total inventory weight whenever items are added or removed. He also ensured that the dwarf's movement speed is updated based on the new total weight.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors confirmed that the total weight is now recalculated correctly after items are dropped, and movement speed adjusts as intended.
- **Issue 3: Lack of Comments and Documentation**
 - **Severity:** Minor
 - **Description:** Some complex functions within the inventory module lacked sufficient comments, making it difficult to understand the code flow and purpose.
 - **Resolution:** Hristian added detailed comments to explain the functionality of complex functions and updated the module's documentation to include descriptions of key classes and methods.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors agreed that the code is now well-documented and more maintainable.

2. Dragon AI Pathfinding Algorithm

- **Contributor:** Brian Rosca
- **Inspectors:** Victoria Rossi, Shaan Kohli, Hristian Tountchev
- **Inspection Dates:** November 20, 2024, and December 2, 2024

Findings:

- **Issue 1: Dragon Occasionally Walks Through Obstacles**
 - **Severity:** Critical
 - **Description:** The dragon sometimes traversed obstacle tiles, which should be impassable, violating game logic and reducing the intended challenge.
 - **Resolution:** Brian reviewed the A* pathfinding implementation and corrected the grid representation to properly mark obstacle tiles as impassable. He updated the pathfinding algorithm to respect these obstacles, ensuring the dragon navigates around them.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors tested various scenarios and confirmed that the dragon no longer walks through obstacles and correctly navigates around them.
- **Issue 2: Performance Issues on Larger Maps**
 - **Severity:** Major
 - **Description:** The dragon's pathfinding caused noticeable lag on larger maps, affecting overall game performance.
 - **Resolution:** Brian optimized the pathfinding algorithm by implementing heuristic functions and limiting the frequency of path recalculations. He also introduced a caching mechanism for paths to reduce computational load.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors observed improved performance on larger maps, with no significant lag during gameplay.
- **Issue 3: Code Readability and Complexity**
 - **Severity:** Minor
 - **Description:** The pathfinding code was complex and lacked sufficient comments, making it difficult for others to understand and maintain.
 - **Resolution:** Brian refactored the code by breaking down large functions into smaller, more manageable ones and added comprehensive comments explaining the logic and flow.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors found the code to be more readable and maintainable.

3. Map Transition and State Management

- **Contributor:** Shaan Kohli
- **Inspectors:** Brian Rosca, Victoria Rossi, Hristian Tountchev
- **Inspection Dates:** November 20, 2024, and December 2, 2024

Findings:

- **Issue 1: No Exit Condition from POIs**
 - **Severity:** Major
 - **Description:** Once the player entered a Point of Interest (POI), there was no mechanism to return to the original map.
 - **Resolution:** Shaan added exit points in each POI, allowing players to return to the main map. He updated the state management system to handle bidirectional transitions, ensuring that the game state is preserved across maps.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors verified that players could now exit POIs and return to the main map seamlessly.
- **Issue 2: State Persistence Issues**
 - **Severity:** Minor
 - **Description:** Upon returning to previous maps, some dynamic elements (e.g., treasure chests, enemy positions) did not retain their state, leading to inconsistencies.
 - **Resolution:** Shaan implemented a more robust state management system that saves and restores the state of each map when the player transitions between them.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors confirmed that map states are now preserved correctly during transitions.
- **Issue 3: Lack of Error Handling**
 - **Severity:** Minor
 - **Description:** The code did not adequately handle exceptions or invalid states, which could potentially lead to crashes.
 - **Resolution:** Shaan added error handling mechanisms to catch exceptions and manage invalid states gracefully. He also implemented logging to assist in debugging any future issues.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors were satisfied with the improved error handling and stability.

4. User Interface (UI) Development

- **Contributor:** Victoria Rossi
- **Inspectors:** Brian Rosca, Shaan Kohli, Hristian Tountchev
- **Inspection Dates:** November 20, 2024, and December 2, 2024

Findings:

- **Issue 1: Unresponsive Buttons**
 - **Severity:** Major

- **Description:** Certain buttons in the menu did not respond to user input under specific conditions, such as after returning from a Point of Interest (POI).
 - **Resolution:** Victoria identified that event listeners were not properly reattached after state changes occurring when exiting a POI. She corrected the code to ensure that event listeners remain active and updated as needed after any state transition.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors tested the buttons in various scenarios and found them to be responsive in all cases.
- **Issue 2: Inconsistent UI Elements**
 - **Severity:** Minor
 - **Description:** Some UI elements, such as fonts and button styles, were inconsistent across different screens, which could lead to user confusion.
 - **Resolution:** Victoria standardized the UI elements by applying a consistent style guide across all screens. She ensured that fonts, button styles, and other UI components were uniform throughout the game.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors confirmed that UI elements were now consistent, enhancing the overall cohesiveness of the interface.
 - **Issue 3: Lack of Navigation Options**
 - **Severity:** Minor
 - **Description:** The UI did not provide clear options for users to navigate back to previous menus or screens, potentially causing confusion.
 - **Resolution:** Victoria added navigation options to the UI screens, enabling users to return to previous menus or the main menu. She ensured that the navigation flow was intuitive and consistent throughout the game.
 - **Re-inspection:** Conducted on December 2, 2024. The inspectors verified that navigation options were implemented correctly, improving the user experience.

V Recommendations and Conclusions

After comprehensive testing and thorough code inspections, the items covered have successfully passed their testing and inspection processes. The collaborative efforts in identifying and resolving issues have resulted in a robust and reliable codebase for the Greedy Dwarf game.

Recommendations:

1. **Implement Continuous Integration (CI):**
 - **Action:** Set up a CI pipeline using tools like Jenkins or GitHub Actions to automate testing and build processes.
 - **Benefit:** Ensures that future code changes are automatically tested, reducing the likelihood of introducing new bugs and facilitating smoother integration.
2. **Enhance Automated Testing Coverage:**
 - **Action:** Develop additional unit tests and integration tests for critical components, including the inventory system, pathfinding algorithm, state management, and user interface.
 - **Benefit:** Improved test coverage will help catch issues early in the development process and maintain code reliability.
3. **Maintain Coding Standards and Documentation:**
 - **Action:** Continue to enforce coding standards across the team and regularly update documentation.
 - **Benefit:** Promotes consistency and makes the codebase easier to understand and maintain for all team members and future contributors.
4. **Schedule Regular Code Reviews:**
 - **Action:** Incorporate regular code reviews into the development workflow.
 - **Benefit:** Ongoing code reviews can help identify potential issues early and encourage knowledge sharing among team members.
5. **Address Remaining Open Issues:**
 - **Action:** Prioritize and resolve the remaining open issues documented in Section VI, such as the chest respawning overlap and inventory management bugs.
 - **Benefit:** Resolving these issues will further enhance the game's quality and player experience.
6. **Plan for Future Enhancements:**
 - **Action:** Evaluate the features listed in the Waiting Room (Section 16) and prioritize them based on impact and feasibility.
 - **Benefit:** Guides future development efforts and keeps the game evolving to meet player expectations.
7. **Gather User Feedback through Beta Testing:**
 - **Action:** Conduct beta testing with a broader audience to gather feedback on gameplay, performance, and user experience.

- **Benefit:** Real-world testing can reveal issues not encountered during internal testing and provide valuable insights for improvements.

VI Project Issues

Open Issues

1. Chest Respawning Overlapping with Dwarf:

- **Description:** When all chests in a level are collected and subsequently respawned, they occasionally appear on the same tiles occupied by the dwarf. This results in both a chest and the dwarf being present on the same tile upon respawn.
- **Impact:** This overlap causes visual clutter and can lead to unintended interactions, such as the dwarf immediately collecting the newly respawned chest or graphical glitches where the dwarf image appears beneath the chest.
- **Status:** Partially addressed. While mechanisms to prevent immediate collection are in place, ensuring that chests do not respawn on the dwarf's current or adjacent tiles remains unresolved.

2. Dragon Occasionally Walks on Obstacles

- **Description:** The dragon antagonist sometimes traverses obstacle tiles (e.g., walls or blocked paths), which should be impassable. This behavior breaks the game's logic and compromises the challenge intended by the enemy AI.
- **Impact:** When the dragon walks on obstacles, it disrupts the dungeon's integrity and diminishes the threat level of the dragon. Players may find it easier to evade the dragon unexpectedly, reducing the game's intended difficulty and satisfaction.
- **Status:** Unresolved. The current pathfinding algorithm does not consistently respect obstacle boundaries, leading to occasional breaches in movement constraints.

3. Inventory Management Bug: Duplicate Item Removal

- **Description:** When a player attempts to remove an item from the inventory that has multiple instances (e.g., two or more of the same item), the system erroneously removes all instances of that item while only adjusting the inventory weight for one instance.
- **Impact:** This bug leads to discrepancies between the actual number of items and the inventory's recorded state. Players may lose items unintentionally,

leading to frustration and a diminished gameplay experience. Additionally, inaccurate weight calculations can disrupt movement speed mechanics.

- **Status:** Unresolved. The inventory management system currently lacks proper handling for multiple instances of the same item, necessitating a more robust implementation.

16 Waiting Room

1. Expanded Character Customization

- **Description:** Introduce additional dwarf characters with unique abilities, such as a dwarf with temporary invincibility or one that can set traps for the dragon.
- **Motivation:** Enhancing character diversity can increase replayability and allow players to experiment with different playstyles.

2. Multiplayer Mode

- **Description:** Develop a cooperative or competitive multiplayer mode where players can team up to collect treasures or compete to achieve higher scores.
- **Motivation:** Adding multiplayer functionality can broaden the game's appeal and foster a community around competitive play.

3. Enhanced Dragon AI

- **Description:** Implement more sophisticated AI behaviors for the dragon, such as evasion tactics, varied pursuit patterns, and adaptive difficulty based on player performance.
- **Motivation:** Improving the dragon's AI can create a more challenging and engaging experience for players.

4. Additional Points of Interest (POIs)

- **Description:** Introduce new POIs with unique themes and mechanics, such as an underwater cavern or a lava-filled chamber.
- **Motivation:** Diversifying POIs can keep the gameplay experience fresh and encourage exploration.

5. Power-Ups and Special Items

- **Description:** Add a variety of power-ups that grant temporary boosts, such as increased speed, invincibility, or enhanced treasure collection rates.

- **Motivation:** Providing power-ups can add strategic depth and excitement to the gameplay.

6. Story Mode

- **Description:** Develop a narrative-driven story mode that provides context and objectives beyond treasure collection, including quests and character development.
- **Motivation:** Incorporating a story can enhance player engagement and provide a more immersive experience.

7. Procedural Soundtrack Integration

- **Description:** Implement a dynamic soundtrack that adapts to the player's actions and the dungeon's environment.
- **Motivation:** A responsive soundtrack can heighten the emotional impact and immersion of the game.

8. Advanced Leaderboard Features

- **Description:** Introduce tiered leaderboards that categorize players based on different criteria, such as fastest escape times or highest treasure values.
- **Motivation:** More detailed leaderboards can encourage diverse competitive goals and player engagement.

17 Ideas for Solutions

1. Preventing Chest Respawn Overlaps

- **Solution Idea:** Implement a collision detection system that checks the dwarf's current and adjacent tile positions before placing respawned chests. If a collision is detected, the chest is placed in the nearest available unoccupied tile.
- **Implementation Suggestion:** Utilize spatial hashing or grid-based checks to efficiently determine valid spawn locations, minimizing performance overhead.

2. Resolving Dragon AI Movement on Obstacles

- **Solution Idea:** Refine the pathfinding algorithm to strictly adhere to obstacle boundaries. Integrate boundary checks that prevent the dragon from calculating paths through impassable tiles.

- **Implementation Suggestion:** Enhance the current A* pathfinding implementation by ensuring that the dragon's movement logic excludes obstacle tiles from potential paths. Additionally, implement unit tests to verify that the dragon never crosses into non-walkable areas.

3. Fixing Inventory Management for Duplicate Items

- **Solution Idea:** Revamp the inventory system to handle multiple instances of the same item accurately. Implement a stack-based inventory where each item instance is individually tracked.
- **Implementation Suggestion:** Modify the inventory data structure to store item counts alongside their properties. When removing an item, decrement the count accordingly without removing all instances. Ensure that the weight calculation reflects the removal of only one instance.

4. Enhancing Interaction Handling in POIs

- **Solution Idea:** Standardize the interaction protocols across all POIs by creating a unified interaction handler that manages treasure and key pickups uniformly, regardless of the POI.
- **Implementation Suggestion:** Develop an interaction manager module that can be easily extended to accommodate new types of interactions, ensuring consistency and scalability.

5. Improving User Feedback Mechanisms

- **Solution Idea:** Introduce tooltips and contextual help within the UI to guide players through complex interactions and game mechanics.
- **Implementation Suggestion:** Utilize UI frameworks that support dynamic element creation and event handling to provide real-time assistance without cluttering the main interface.

6. Optimizing Inventory Weight Calculations

- **Solution Idea:** Implement unit tests for the inventory management system to verify that weight calculations are accurate under various scenarios, such as adding, removing, and dropping items.
- **Implementation Suggestion:** Use a testing framework like Jest to create automated tests that validate the correctness of inventory operations, ensuring reliability and preventing regressions.

7. Enhancing Map State Management

- **Solution Idea:** Refactor the map state management to separate player positioning from map entities, ensuring that updates to the map do not inadvertently affect the player's current position.
- **Implementation Suggestion:** Utilize state management libraries or design patterns that facilitate clear separation of concerns, such as Redux or the Observer pattern, to maintain consistent and reliable map states.

18 Project Retrospective

The Greedy Dwarf project has been quite a challenge to develop, but very rewarding with huge learning experiences for our team. One of the biggest strengths in the whole project was effective collaboration. Regular meetings, clear communication, and defined roles allowed us to align efforts and make use of the strengths each individual brings to the table in an efficient manner. Detailed planning and documentation gave a solid base, further aided by project management tools in organizing tasks and providing accountability. Following coding standards and regular code reviews improved code quality and facilitated knowledge sharing.

However, we did face some notable challenges, from which valuable lessons were learned. Time management was a big challenge; the underestimation of complexities in tasks led to tight deadlines and increased pressure. Balancing project work with other academic responsibilities sometimes resulted in uneven workload distribution. Scope creep also affected our efficiency; introducing new features during development diverted focus from core functionalities and strained resources.

In order to improve future projects, we recognize the need for better time management by allowing more realistic time allocations and scheduling regular progress checks. This will help maintain focus on the essential components by having stricter scope control in place, such as a feature freeze point and proper task prioritization. Our testing practices can also be improved by incorporating automated testing and adopting test-driven development to make our code more efficient and reliable. Also, the team will be able to enhance collaboration and fill in gaps of skills by encouraging consistent use of communication tools and knowledge sharing.

This could apply to the Greedy Dwarf project, which brought us together and brought out certain strengths and areas to improve. Looking back at this experience, we feel enthused to put into action these lessons for improved efficiency, product quality, and general performance of a team that is ready for the next project with still great confidence and success.

VII Glossary

A* (A-Star) Pathfinding Algorithm: A popular algorithm used in artificial intelligence and game development to find the shortest path between two points.

Apple iOS: An operating system designed by Apple for their mobile devices such as iPhones and iPads.

Arcade-Action Genre: A style of video game focused on quick reflexes, simple controls, and short play sessions. Classic examples include titles like Pac-Man or Space Invaders.

Character Class: A category or type of character that defines its attributes, abilities, and gameplay style.

Crossroads City: One of the Points of Interest (POIs) in Greedy Dwarf. This area offers unique treasures, terrain, and challenges different from other locations in the game.

Google Play Store: An online platform for downloading apps and games for Android devices. Greedy Dwarf can be made available here for mobile play.

Greedy Dwarf: The name of the game described in the report. Players control a dwarf who must navigate dungeons, collect treasures, and avoid a dragon.

Heuristic Function: In pathfinding, a heuristic function estimates how close you are to the goal. A* uses heuristics to help the dragon find the shortest path to the dwarf efficiently.

IEEE Standard (IEEE Std 1028-1997): A recognized set of guidelines for conducting software reviews and audits. These standards help ensure quality and consistency in software development processes.

Inventory: A system that stores all items the player has collected (such as treasures and keys). The dwarf's movement speed can decrease as the inventory becomes heavier.

Knapsack Problem: A well-known mathematical and computational problem where you must choose items of varying weights and values to maximize total value without exceeding a weight limit. In Greedy Dwarf, this concept inspires how players manage which treasures to carry.

Leaderboard: A feature that displays the top scores achieved by players locally or globally, encouraging competition and replayability.

Linux: An open-source operating system often used on PCs and servers. Greedy Dwarf can be run on Linux as one of its supported platforms.

Manhattan Distance: A way of measuring distance in a grid by counting the number of steps horizontally and vertically between two points. It was initially used by the dragon's simpler AI before upgrading to A*.

Map Matrix: A two-dimensional array representing the game's environment, where each element corresponds to a type of tile (e.g., path, wall, treasure, exit).

Miner Dwarf: A character class with a high carrying capacity but slower movement speed. This class can hold more treasure but moves less quickly.

Non-Functional Requirements: Specifications detailing the system's qualities, such as performance, usability, and reliability, rather than features or behaviors.

Non-Player Character (NPC): A character in the game not controlled by the player. The dragon is an example of an NPC that chases the player.

Pac-Man: A classic arcade game that partially inspired Greedy Dwarf's top-down, maze-like gameplay: The process by which characters (like the dragon) determine how to move through the game world efficiently. A* is an example of a pathfinding algorithm style.

Pathfinding: The process by which characters (like the dragon) determine how to move through the game world efficiently. A* is an example of a pathfinding algorithm.

Points of Interest (POI): Special areas in Greedy Dwarf, like Jungle City, Dragon's Lair, and Crossroads City, each offering unique treasures, layouts, and challenges.

Procedurally Generated (or Procedurally Loaded): A method of creating game content algorithmically rather than manually, resulting in unique and varied environments each time the player enters.

React: A popular JavaScript library for building user interfaces. Greedy Dwarf uses React features such as hooks and callbacks to manage the game's state and UI efficiently.

Regression Testing: Re-running tests after fixes or new features are implemented to ensure that previously working functionalities still operate correctly, and no new bugs have been introduced.

Scout Dwarf: A character class that moves quickly but cannot carry as much weight. Ideal for players who prefer speed and agility over heavy treasure collection.

Steam: A digital distribution platform for purchasing and downloading games on PCs. Greedy Dwarf may be offered here for computer gamers.

User Interface (UI): The set of on-screen elements through which the player interacts with the game, including menus, score displays, and inventory screens.

Unified Modeling Language (UML): A standardized system for visually modeling a software's structure and behavior. UML diagrams help developers plan and communicate their designs.

User: The person playing the game. Also referred to as the "player."

User Authentication: A system that identifies and verifies players, allowing them to track their progress and scores under a chosen username.

Warrior Dwarf: A balanced character class offering moderate speed and carrying capacity, suitable for players who want a more even playing style.

Weight: A measure of how heavy a collected item is. In Greedy Dwarf, carrying heavier items slows the dwarf down, affecting the player's strategy.

Windows: A widely used operating system on PCs. Greedy Dwarf can be played on Windows-based machines.

Value: The worth of a collected item in terms of score. Higher-value items typically weigh more but yield greater point rewards.

Valve: A well-known video game company and digital distribution service operator (creator of Steam) that sets various standards for distributing and playing games online.

VIII References / Bibliography

- [1] A. Lau, C. Rivera, M. Searcy, Greedy Dwarf, 2019.

IX Index

<i>Glossary</i>	55	<i>Project Description</i>	4
<i>Index</i>	59	Naming Conventions and Definitions.....	6
<i>Inspection</i>	42	Project Domain	4
Inspection Procedures	43	<i>Project Overview</i>	4
Inspection Results	45	Relationship to Other Documents.....	5
<i>Items to be Inspected</i>	42	<i>Project Issues</i>	50
<i>List of Figures</i>	3	Ideas for Solutions	52
<i>List of Tables</i>	3	<i>Open Issues</i>	50
<i>Naming Conventions and Definitions</i>		Project Retrospective	54
Data Dictionary for Any Included		<i>Waiting Room</i>	51
Models.....	7	<i>Recommendations and Conclusions</i>	49
Definitions of Key Terms.....	6	<i>References / Bibliography</i>	58
UML and Other Notation Used in This		<i>Table of Contents</i>	2
Document.....	7	<i>Testing</i>	15
<i>Project Deliverables</i>	10	<i>Items to be Tested</i>	15
Comparison with Original Project Design		Regression Testing.....	39
Document.....	15	Test Results	31
First Release.....	10	Test Specifications	16
Second Release	12		